

AI Engineer Test Task

AI Engineer (Conversational Agents) — Test Task

Developer Test Task: Conversational Meal Logging Agent

Overview

Role: AI Engineer — Conversational Agents **Company:** CalorAI **Time Budget:** 6–8 hours maximum **Deadline:** Complete at your own pace (suggested: within 2 days of starting)

Why We Use Test Tasks

We believe in "working together before working together." This test task lets you showcase how you design and ship agent systems while giving us a fair, objective way to evaluate all candidates equally. Everyone receives the same project — no tricks, no gotchas.

Important: We heavily encourage and expect AI tool usage (Claude Code, Cursor, Copilot, ChatGPT, etc.). We value implementation speed and engineering judgment over writing everything manually. Treat AI as your coding partner.

Project Brief

What You're Building

CalorAI Logging Agent: A conversational agent that logs meals the way people actually talk about food. CalorAI is a WhatsApp-based calorie tracker. There are no forms, no dropdowns, no searching a database for "paratha (medium)." You text what you ate, like you'd text a friend, and it gets logged. That's the whole bet. It's also the hard part. People don't describe food cleanly — they send half-sentences, photos, corrections, and references to things they ate last week. Your agent has to handle all of it and answer fast enough that it still feels like messaging.

User Story

As a CalorAI user, I want to log my meals by texting or sending photos in plain language, so that I can track my nutrition without ever opening a form.

Detailed Requirements

Core Features (Must Have)

- 1. Conversational agent with tool calling** Built in LangChain or LangGraph. At minimum the agent needs tools to log a meal, retrieve past meals, look up nutrition data, and return current totals. How you split up the tool surface is your call and is part of what we're evaluating.
- 2. Persistent database** Postgres, SQLite, or Supabase. Schema is your call. Meals must persist across turns and across sessions.

3. Running daily totals Calories and macros for the current day, updated as meals come in, correctly reflecting edits and deletions. The user should be able to ask "how am I doing today?" at any point and get an accurate answer.

4. Image input on a separate model A user sends a photo of their plate; the agent logs it. Route images to a vision model of your choice and text to whatever you're using for conversation — do not run everything through one model. Document why you picked each and how you handle the handoff, including what happens when the vision model returns something ambiguous or wrong.

5. Persistent memory The agent should remember things worth remembering across sessions — that this user is vegetarian, that "my usual" means a specific breakfast, that they're targeting 140g of protein. You decide what's worth storing, how it gets written, and how it gets retrieved into context without bloating every prompt. **This is the part we're most interested in.**

6. Multi-turn ambiguity handling The agent decides when it has enough information to log and when it needs to ask. Over-asking kills the experience; under-asking produces garbage data. Where you draw that line is a product decision as much as a technical one.

Nice-to-Have (Bonus Points)

- LangSmith tracing and monitoring wired up
- An eval set with a defined notion of what "correct" means for this task
- Streaming responses
- Handling of multiple users / session isolation
- Anything else you think the problem genuinely needs

Note: Focus on core features first. Only tackle bonuses if you have time remaining.

Latency Requirements

This runs on WhatsApp. A user texting "had 2 rotis" expects a reply at messaging speed, not after a ten-second agent loop. Adding image handling makes this harder, which is the point.

You must measure and report p50 and p95 response times for both the text path and the image path.

Explain what you did to get there — what you cut, what you parallelised, where you chose a faster path over a more thorough one, what you cache. If something is slow and you couldn't fix it, say so and say why.

We care more about the reasoning than the number.

Technical Specifications

Required Tech Stack

- **Python**
- **LangChain or LangGraph** for the agent
- **Any LLM provider** (OpenAI, Anthropic, Gemini, Bedrock, Vertex — your choice, document why)
- **Any database** (Postgres / Supabase / SQLite)

Suggested but optional

LangSmith, pgvector or any vector store, Pydantic for structured outputs, FastAPI if you want an HTTP surface.

Technical Constraints

- Must run locally from a clean clone with documented setup
 - Must accept image input (a file path or upload is fine — you do not need a WhatsApp integration)
 - No authentication required
 - No frontend polish required
 - Nutrition data can come from a real API, a small hardcoded table, or the model itself — document your choice and its tradeoffs
-

Resources Provided

1. Test Conversation Set

Your agent should handle all of the following. These are drawn from real user messages.

```
"had 2 parathas and chai for breakfast"  
"leftover biryani, maybe two thirds of the box"  
"skipped lunch but grazed all afternoon"  
"same as yesterday"  
"actually that was 3 rotis not 2"  
"how much protein have I had today?"  
"how am I doing on calories?"  
[photo of a plate]  
[photo of a plate] "half of this was my brother's"  
"my usual"  
"i'm vegetarian btw"
```

Three of these are the ones that separate good submissions from average ones:

- **"actually that was 3 rotis not 2"** — totals must update, not double-count
- **"same as yesterday" / "my usual"** — requires memory, not parsing
- **[photo] + caption** — both models must resolve to one meal, not two

2. Interface

CLI is fine, as long as it accepts an image path. A simple chat UI is fine. Do not spend time on frontend — we are reading the agent code.

Definition of Done

Your submission is complete when:

- Agent runs from a clean clone with documented setup
- All six core features are implemented
- Meals persist across sessions
- Daily totals stay correct through edits and corrections
- Image input works and routes to a separate vision model
- Memory persists across sessions and is actually used in responses
- Latency numbers are measured and reported

- README covers all required sections
 - Walkthrough video is recorded (5–10 minutes)
-

Submission Requirements

1. GitHub Repository

- Clean commit history (show your work progression)
- Descriptive commit messages
- No single "final commit" with all code

[README.md](#) must include:

- Project overview
- Setup / installation instructions
- Model choices and why (text model, vision model)
- How memory works — what you store, when you write, how you retrieve
- Tool design and why you split it that way
- Latency numbers (p50 / p95, text and image) and what you did to optimise
- Assumptions and trade-offs you made
- Time breakdown (how you spent your hours)
- What you'd fix or build next with more time
- Notes on AI tool usage (which tools, how they helped)

2. Walkthrough Video (5–10 minutes)

- Demo the working agent, including at least one image case and one correction case
- Explain your architectural decisions
- Walk through how memory is written and retrieved
- Discuss the latency work and the tradeoffs you made
- Cover any challenges and how you solved them
- Show any bonus features you implemented

3. [Optional] LangSmith Trace Link

If you set up tracing, share a public trace link in the README.

How to Submit

Fill out the attached Google form with:

- GitHub repo link
- Video link (Loom, YouTube unlisted, or Google Drive)
- Any questions or notes about your submission

Form:

forms.gle

https://forms.gle/1mxRopiMwKsNG6NQ9

Evaluation Criteria

Area	What we look for
Conversation quality	Does it feel like texting a friend, or like filling out a form with extra steps
Agent architecture	Sensible tool boundaries, clean state handling, sane control flow
Memory design	What you chose to remember and forget, and how it gets into context efficiently
Data correctness	Totals stay right through corrections, edits, and ambiguity
Multimodal handling	Clean separation of models, graceful handling of vision uncertainty
Latency thinking	Measured, reasoned about, honest about what you couldn't fix
Code quality	Clear structure, good separation of concerns, readable
Process	Iterative commit history, clear README, honest time management
Communication	Clear video walkthrough and documented decisions

Green Flags

- Tool boundaries that make sense and don't overlap confusingly
- Memory that's actually selective — not "store every message and hope"
- Handles corrections without double-counting
- Vision model uncertainty surfaced to the user instead of silently guessed
- Measured latency with real numbers and real tradeoff reasoning
- Evals set up, even a small one
- Clean commit history showing iterative development
- Honest README about what's incomplete
- Smart use of AI tools (documented)

Red Flags

- Doesn't run from a clean clone
 - Missing core features with no explanation
 - Everything through one model, including images
 - "Memory" that's just conversation history stuffed into the prompt
 - Totals that break on a correction
 - No latency measurement at all
 - One giant commit with all code
 - No documentation
 - Overbuilt: a Kubernetes deploy and no working agent
-

Communication Guidelines

Questions Welcome

If something is unclear, ask. We'd rather you clarify than assume incorrectly.

Contact: Bhavesh at bhavesh@calorai.ai

Progress Updates (Optional but Appreciated)

Feel free to send a quick update:

- When you start
 - If you hit a major blocker
 - When you submit
-

FAQs

Q: What if I can't finish in 8 hours? A: Submit what you have. Document what's incomplete and what you'd do next. We value honest time management over heroic overtime. We'd genuinely rather see four things done well than six done badly.

Q: Can I use AI tools (Claude Code, Cursor, Copilot, ChatGPT)? A: YES. We heavily encourage it. This is about building fast and well, not proving you can code without help. Just document which tools you used in your README.

Q: Which LLM should I use? A: Your choice. Part of what we're evaluating is whether you can justify the pick — cost, latency, and quality all matter here, and the right answer is different for the text path and the vision path.

Q: Do I need to build a WhatsApp integration? A: No. A CLI or simple chat interface is fine. Just make sure it accepts images.

Q: How accurate does the nutrition data need to be? A: Not very. We're not evaluating your nutrition database. We're evaluating how the agent handles ambiguity, memory, and conversation. Hardcoded or model-generated values are fine — just document the choice.

Q: What counts as "memory"? A: That's for you to decide and defend. Conversation history is not memory. We're looking for something that survives across sessions and gets selectively retrieved — how you build it is the interesting part.

Q: Should I write unit tests? A: A few around your tools would be a green flag, but this isn't a testing exercise. If you have limited time, spend it on evals rather than unit tests.

Q: Do I need a frontend? A: No. CLI is completely fine.

Q: Can I change the requirements if I think something's wrong? A: Implement what's specified, but if you think a requirement is a mistake, say so in the README. We'd rather hire someone with opinions.

Ready to Start?

1. Reply to confirm you're starting
2. Review the test conversation set above
3. Build something great
4. Submit before you hit 8 hours (or document why it took longer)

Good luck. We're excited to see what you build.